

SILIGENT

Dental AI Assistant

Engineering & Operations Handoff

Setup, operation, feature usage, and developer reference for new project owners

Document	Product & Engineering Handoff
Product	Siligent Dental AI Assistant
Version	2.0
Date	June 13, 2026
Audience	Incoming owners, operators, and engineers
Classification	Confidential, Internal Use Only

This document describes software that handles protected health information (PHI). It is intended for local, private deployment and authorized personnel only.

Contents

- 1. Product Summary 3
- 2. Repository Map 3
- 3. Runtime Architecture 4
- 4. Environment Configuration 5
- 5. Starting and Stopping 6
- 6. Authentication and Roles 7
- 7. How to Use Each Feature 8
 - 7.1 Denial Letter Generation 8
 - 7.2 Insurance Verification 9
 - 7.3 Email Exchange Drafting 11
 - 7.4 Template Library 12
 - 7.5 Template Draft Generation 13
 - 7.6 Model Settings 13
 - 7.7 System Health and Audit 14
- 8. Prompt and Generation System 14
- 9. Data and Persistence 15
- 10. API Surface 15
- 11. Developer Code Map 17
- 12. Verification Checklist 22
- 13. Known Operational Constraints 23
- 14. Maintenance Notes 23
- 15. Recent Changes 25

1. Product Summary

The Siligent Dental AI Assistant is a local-first, dental front-office AI application. It helps administrative users generate and review dental communications using local Ollama models, structured templates, uploaded context, and controlled runtime fields.

Current Production Stack

Layer	Technology
Frontend	React + TypeScript + Vite, served through Nginx in Docker.
Backend	FastAPI.
Database	PostgreSQL with pgvector.
Model runtime	Ollama running on the host machine.
Deployment	Docker Compose for frontend, api, and db. Ollama is not containerized.
Primary roles	admin and staff.

Current Product Workflows

- Insurance denial letter generation.
- Insurance verification.
- Email exchange reply drafting.
- Template library management.
- Model routing and settings.
- System health and audit review.

2. Repository Map

The most important locations in the repository are listed below.

Path	Purpose
frontend/	React application.
frontend/src/pages/	Main UI pages.
frontend/src/components/	Reusable UI components.
frontend/src/lib/	Frontend API client, auth, formatting, runtime field helpers, and task state.
api/main.py	FastAPI app, route registration, auth dependencies, and request handlers.
api/schemas.py	API request and response schemas.

services/	Backend business logic, storage, model calls, prompt handling, generation, and guardrails.
prompts/	Version-controlled prompt files used by the app.
data/payer_references/	Local payer reference files used for insurance verification.
docs/	Product-specific supporting documents and field-mapping specs.
docker-compose.yml	Runtime composition for database, backend, and frontend.
start.sh	Starts the Dockerized app stack after checking host Ollama.
stop.sh	Stops the Dockerized app stack and attempts to offload Ollama models.
installer.sh	Installer-style dependency, environment, port, and runtime checker.
.env.example	Environment variable template.
requirements.txt	Python dependencies.
frontend/package.json	Frontend dependencies and build scripts.
tests/	Backend test suite.

3. Runtime Architecture

The browser loads the React app from `http://localhost:3000`. The React app calls the FastAPI backend at `http://localhost:8000/api`. The backend stores structured application data in PostgreSQL and stores uploaded file content under `data/uploads` through the mounted `./data:/app/data` volume.

Ollama runs directly on the host at `http://localhost:11434`. Inside Docker, the API reaches Ollama through `http://host.docker.internal:11434`.

Runtime Services

Service	Technology / Host Port	Notes
db	PostgreSQL 16 with pgvector — 5434	Structured application data
api	FastAPI backend — 8000	REST API served under /api
frontend	React static app via Nginx — 3000	Single-page application
ollama	Host process (not containerized) — 11434	Local model runtime

4. Environment Configuration

Create a working environment file from the template:

```
cp .env.example .env
```

The most important variables are summarized below.

Variable	Default / Purpose
OLLAMA_URL	Host Ollama URL. Compose overrides this to <code>host.docker.internal:11434</code> .
MODEL_NAME	<code>qwen3.5:4b</code> — Default local model.
OLLAMA_GENERATE_TIMEOUT_SEC	180 — Generation timeout, in seconds.
OLLAMA_NUM_PREDICT	1024 — Maximum model output tokens.
OLLAMA_THINK	<code>false</code> — Whether to use model thinking mode.
OLLAMA_KEEP_ALIVE	0 — Ollama model keep-alive. 0 unloads the model after use.

AUTH_ENABLED	true — Authentication toggle.
AUTH_SESSION_HOURS	12 — Session duration, in hours.
ALLOW_SELF_REGISTER	false — Self-registration toggle.
API_KEY	blank — Optional API key gate. Blank disables the API key check.
CORS_ORIGINS	http://localhost:3000 — Allowed frontend origin.
DATABASE_URL	Postgres connection string.
POSTGRES_*	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_PORT compose database settings.

Production note. For deployment beyond local development, change the default database credentials before starting the stack.

5. Starting and Stopping

Prerequisites

- Docker Desktop or Docker Engine is running.
- Ollama is installed on the host.
- The selected model is pulled locally.

Pull the Default Model (if needed)

```
ollama pull qwen3.5:4b
```

Start the App

```
./start.sh
```

Installer-Style Setup / Check

The `installer.sh` script checks for and can help install: `curl`, `Docker`, `Docker Compose` plugin, and the `Ollama CLI`. On macOS it uses `Homebrew`; on Linux it uses `apt-get`. It checks whether the `Docker` daemon is running and whether the `Ollama` server is reachable. The installer does not pull the model — that step is done manually.

```
./installer.sh ./installer.sh --check-only ./installer.sh --no-start
```

Typical Clean Setup

```
git clone <repository-url> cd dental ollama pull qwen3.5:4b ./installer.sh
```

Stop the App

```
./stop.sh
```

Access Points

Component	Address
App	http://localhost:3000
API health / docs	http://localhost:8000/docs
Database	localhost:5434
Ollama	http://localhost:11434

6. Authentication and Roles

Authentication is enabled by default.

Bootstrap Behavior

1. Open `http://localhost:3000`.

2. If no users exist, the app shows the registration / bootstrap experience.
3. Create the first user with a username and password.
4. The first user becomes the admin account.
5. After bootstrap, users log in through the login screen.

Roles

Role	Capabilities
admin	Can manage shared templates, template types, model preferences, users, field dictionary entries, and audit / system areas.
staff	Can use generation workflows and personal templates, but cannot manage shared system-level configuration.

Operator Rule

- Create one admin owner first.
- Use admin access to create staff accounts.
- Do not share admin credentials with day-to-day staff users.

7. How to Use Each Feature

7.1 Denial Letter Generation

Purpose. Use this flow when an insurance claim has been denied and staff need a professional appeal or denial-response letter from the Siligent provider / front-office perspective.

Who should use it

- Staff users can generate and edit letters.
- Admin users can also manage the shared denial letter template used by the workflow.

Before starting, confirm:

- The payer name.
- The patient name.
- The date of service.
- The claim / reference number, if available.
- The procedure description and procedure code.
- The denial code and denial reason.
- The appeal basis — the specific reason Siligent is asking the payer to reconsider.

Step-by-step

1. Log in to the app.
2. Open the denial letter workflow.
3. Confirm the workflow is for denial letters.
4. Enter patient details exactly as they should appear in the letter.
5. Enter payer details. If the payer address is unknown, do not invent one.
6. Enter the denial code and denial reason from the EOB or payer notice.
7. Enter the procedure description and procedure code.
8. Enter the appeal basis in plain English. For example, "The procedure was medically necessary due to documented infection and symptoms."
9. Generate the draft.
10. Review the full letter before use.
11. Edit the draft if wording needs to be adjusted.
12. Copy, download, or print / save as PDF after human review.

Review checklist

- No invented patient names, dates, providers, phone numbers, payer addresses, claim IDs, or procedure codes.
- Denial reason and appeal basis are logically connected.
- The tone is professional and administrative, not casual or AI-narrated.
- The letter is ready for a human staff member or provider to approve.

Common issues

- If the letter says Not provided, add the missing field and regenerate.
- If the appeal rationale is generic, improve the appeal basis field and regenerate.
- If the letter includes facts not entered by the user or not present in the source context, remove them before use.

7.2 Insurance Verification

Purpose. Use this flow to summarize locally stored payer reference information and determine whether a requested condition / procedure appears covered, not covered, or needs manual review based on available local policy text.

Step-by-step

1. Log in to the app.
2. Open the insurance verification workflow.
3. Select the payer.
4. Enter patient / member details.
5. Enter the requested condition. For example, "symptomatic irreversible pulpitis" or "tooth infection."
6. Enter the requested procedure. For example, "root canal therapy" or "D3310 anterior RCT."
7. Generate the verification.
8. Review the structured output.
9. Read the coverage verdict and rationale.
10. Use the result as a staff aid, then confirm with payer systems or office policy if the result is high-impact.

How to interpret the verdict

Verdict	Meaning
Covered	The local payer reference supports coverage for the requested procedure / context.
Not covered	The local payer reference indicates exclusion or no coverage.
Manual review needed	The local payer reference is incomplete, ambiguous, or does not clearly answer the specific question.

Review checklist

- The verdict is tied to local payer reference data.
- The result does not claim real-time payer eligibility.
- The output does not invent plan benefits absent from the payer reference.
- The user understands this is not a live payer transaction.

7.3 Email Exchange Drafting

Purpose. Use this flow to draft a personalized reply to a pasted or uploaded email thread. The app does not connect directly to a mailbox. The user provides the thread context manually.

Step-by-step

1. Log in to the app.
2. Open the email exchange workflow.
3. Paste the email thread text or upload relevant context (up to 3 files).
4. Enter recipient name or role, if known.
5. Enter sender or office role, if needed.
6. Enter the topic of the reply.
7. Enter the intended next step.
8. Optionally select a preferred email intent from the supported list.
9. Generate the draft.
10. Review the response for accuracy and tone.
11. Edit the draft directly before sending it in the actual email system.
12. Copy the final approved text into the mailbox or external system.

Supported email intents

- appointment_confirmation
- insurance_update
- billing_inquiry
- records_request
- post_treatment_followup
- denial_followup
- general_inquiry

Review checklist

- The response answers the actual latest email in the thread.
- The tone is appropriate for dental front-office communication.
- No patient facts, dates, phone numbers, or insurance details were invented.
- The draft is edited and approved before being sent externally.

7.4 Template Library

Purpose. Use this area to manage reusable wording for letters, emails, and workflow-specific templates. Templates use `[[Variable Name]]` placeholders for patient- and case-specific data.

- Admin users manage shared canonical templates and template types.
- Staff users can use templates and save personal variants where permitted.
- Templates should not contain real PHI — use placeholders for all patient-specific values.

7.5 Template Draft Generation

Purpose. Use this when an admin wants the model to draft a starter template that can then be edited and saved by a human.

1. Open the template creation or template library workflow.
2. Choose the template type, or create the needed type if allowed.
3. Provide the intended purpose and required fields.
4. Generate a template draft.
5. Review the draft carefully — the model will automatically repair drafts that contain literal values instead of placeholders.
6. Edit wording until it fits the office's preferred tone.
7. Save as the canonical shared template if ready for staff use.
8. Run one generation test using realistic runtime fields.

7.6 Model Settings

Purpose. Use this area to control which locally installed Ollama model is used globally or for specific use cases. Admin users only.

1. Confirm the desired model is installed locally with ollama list.
2. Log in as admin.
3. Open Model Settings.
4. Choose a global model or configure per-use-case model routing.
5. Save settings.
6. Run a small generation test for each affected workflow.

Use cases that can be routed independently: denial_letters, email_drafting, email_thread, insurance_verification, document_ingestion, template_authoring.

7.7 System Health and Audit

Purpose. Use this area to confirm the app is operating and to review administrative audit events. Admin users only.

Troubleshooting order

1. Run docker compose ps.
2. Check API logs with docker compose logs -f api.
3. Check frontend availability at http://localhost:3000.
4. Check API availability at http://localhost:8000/.
5. Check Ollama with ollama list and ollama ps.
6. Restart with ./stop.sh then ./start.sh if needed.

8. Prompt and Generation System

Prompt files live under `prompts/`. Prompt routing is centralized in `services/prompt_registry.py`.

Generation-Related Services

Service	Responsibility
<code>services/generation.py</code>	Denial letters, email drafts, insurance verification, and template drafts. Includes placeholder-safe template repair logic.
<code>services/prompt_engine.py</code>	Prompt loading and rendering helpers.
<code>services/template_runtime.py</code>	Runtime field normalization, alias expansion, and template rendering.
<code>services/document_pipeline.py</code>	Multi-document extraction and Smart Composer support.
<code>services/email_thread.py</code>	Thread analysis, intent detection, template recommendation, and reply generation.
<code>services/email_guardrails.py</code>	Email role/purpose enforcement. Validates that drafts use practice voice, match the declared intent, and contain no unresolved placeholders. Supports automatic rewrite on non-compliance.
<code>services/autonomy_policy.py</code>	Factual grounding controls. Raises <code>UNGROUND_FFACTS</code> error when generated output contains verifiable values (dates, IDs, codes) not present in trusted input sources.
<code>services/ollama_client.py</code>	Ollama health, model listing, and generation API client.

Operational Rule. Prompt text should remain in `prompts/**/* .txt`. Runtime code should select prompts through the prompt registry. Do not scatter prompt strings directly across UI or route handlers.

9. Data and Persistence

Primary persistence is PostgreSQL, accessed through `DATABASE_URL`.

Postgres-Backed Stores

- Users and sessions.
- Templates (with hash-based vector embeddings for semantic reranking).
- Template types.
- Field dictionary (canonical field keys, labels, and aliases).
- Model preferences.
- Upload metadata.
- Audit events.

Filesystem Data

Path	Contents
------	----------

data/payer_references/	Payer coverage reference text files.
data/uploads/	Uploaded source files and extracted document content.
prompts/	Read-only prompt templates mounted into the API container.

The field dictionary stores canonical runtime field keys, human-readable labels, and aliases. It is used by `expand_runtime_field_aliases` in `services/template_runtime.py` to normalize user-supplied field names before template rendering.

10. API Surface

The primary endpoints, grouped by functional area, are listed below. API errors are returned as structured JSON with an error code and a human-readable message.

Area	Method	Endpoint
System	GET	/
System	GET	/api/health
Auth	GET	/api/auth/bootstrap
Auth	POST	/api/auth/register
Auth	POST	/api/auth/login
Auth	POST	/api/auth/logout
Auth	GET	/api/auth/me
Models	GET	/api/models
Models	GET	/api/model-preferences
Models	PUT	/api/model-preferences
Template types	GET	/api/template-types
Template types	POST	/api/template-types
Field dictionary	GET	/api/field-dictionary
Field dictionary	PUT	/api/field-dictionary/{field_key}
Field dictionary	DELETE	/api/field-dictionary/{field_key}
Reference data	GET	/api/denial-codes
Reference data	GET	/api/payers
Reference data	GET	/api/email-scenarios
Generation	POST	/api/denial-letters/generate
Generation	POST	/api/emails/generate
Generation	POST	/api/email-thread/generate
Generation	POST	/api/insurance-verification/generate

Generation	POST	/api/document-pipeline/generate
Generation	POST	/api/templates/generate-draft
Dentrix	POST	/api/dentrix/resolve-template-fields
Templates	GET	/api/templates
Templates	POST	/api/templates
Templates	DELETE	/api/templates/{index}
Audit	GET	/api/audit-events

11. Developer Code Map

Use this section when handing the product to an engineer who needs to understand where each workflow is implemented.

Backend Entry Points

`api/main.py`

- `create_app` creates the FastAPI app.
- `register_error_handlers` maps exceptions into structured JSON errors.
- `register_routes` defines all API routes.
- `require_api_key`, `require_current_user`, and `require_admin` enforce access control.
- `_resolve_model_for_use_case` chooses the Ollama model for a workflow.
- `_parse_runtime_fields`, `_prepare_runtime_values`, and `_merge_extracted_runtime_values` prepare user / runtime fields before generation.
- `_normalize_requested_email_intent` validates the declared intent against the allowed-intents set before email-thread generation.

`api/schemas.py`

Contains request / response models for auth, templates, denial letters, insurance verification, email threads, document pipeline, audit events, and Dentrrix mapping.

Denial Letter Workflow

Frontend

`frontend/src/pages/SmartComposerPage.tsx` — handles denial-letter form state, runtime fields, draft display, editing, and actions.

Backend route

POST `/api/denial-letters/generate` in `api/main.py`.

Backend services

- `services/generation.py` — `generate_denial_letter` renders the denial prompt and calls Ollama.
- `services/prompt_registry.py` — `resolve_denial_prompt` maps a denial code to a prompt path.
- `services/constants.py` — stores supported denial codes and descriptions.
- `services/autonomy_policy.py` — `enforce_draft_grounding_or_raise` checks generated drafts for ungrounded factual values.

Prompts

`prompts/denial_letters/*.txt`

Insurance Verification Workflow

Backend route

POST `/api/insurance-verification/generate` in `api/main.py`.

Backend services

- `services/generation.py` — `list_available_payers`, `load_payer_reference`, and `generate_insurance_verification` build the prompt, call Ollama, and return structured output.
- `services/verification.py` — `extract_json_object`, `normalize_verification_summary`, and `enforce_grounded_verification_summary` parse and validate the output.

Prompt / data

`prompts/insurance_verification.txt` and `data/payer_references/*.txt`

Email Exchange Workflow

Backend route

POST /api/email-thread/generate in api/main.py.

Backend services

- services/email_thread.py — normalize_email_thread, extract_latest_message, build_email_thread_context, analyze_email_thread_with_model, recommend_email_templates, generate_email_thread_reply, and ensure_email_reply_has_no_missing_markers.
- services/email_guardrails.py — build_enforced_email_prompt injects role/purpose rules. generate_with_guardrails generates and optionally rewrites non-compliant drafts (up to 2 rewrite attempts for email-thread). normalize_purpose_label normalizes the intent string before validation. is_role_and_purpose_compliant validates structure and purpose.

Prompts

prompts/email_thread/analyze_thread.txt, prompts/email_thread/generate_reply.txt, prompts/emails/*.txt

Template Library and Template Drafting

Backend routes

GET /api/templates, POST /api/templates, DELETE /api/templates/{index}, GET /api/template-types, POST /api/template-types, POST /api/templates/generate-draft

Backend services

- services/template_store.py and services/postgres_store.py store templates.
- services/postgres_store.py — rerank_with_context performs semantic reranking of recommended templates using SHA-256 hash-based vector embeddings (384 dimensions, pgvector). Called when the postgres store is active.
- services/template_type_store.py normalizes and manages template types.
- services/default_templates.py seeds missing shared default templates on startup.
- services/template_runtime.py — extract_template_placeholders, normalize_runtime_fields, expand_runtime_field_aliases (uses field dictionary), render_template_with_runtime_fields, and runtime_fields_to_json_context.
- services/generation.py — generate_template_draft creates model-assisted reusable template drafts. _repair_template_to_placeholder_only auto-repairs drafts that contain literal values instead of [[Variable Name]] tokens.

Document Upload and Smart Composer Internals

Backend route

POST /api/document-pipeline/generate in api/main.py.

Backend services

- `services/document_pipeline.py` — `extract_document_content` (validates, extracts text and images), `build_document_context` and `build_document_generation_context` prepare prompt context, `heuristic_detect_template_type` and `detect_template_type_with_model` classify document purpose, `recommend_templates` scores template matches, `generate_structured_output` produces structured draft output, `stabilize_structured_output` repairs weak or low-signal drafts, `_repair_final_draft` makes a second model pass on structurally invalid output.
- Supported file types: `.txt`, `.eml`, `.md`, `.csv`, `.json`, `.log`, `.xml`, `.yaml`, `.yml`, `.rtf`, `.docx`, `.pdf`, `.png`, `.jpg`, `.jpeg`, `.heic`, `.bmp`, `.tif`, `.tiff`, `.webp`, `.gif`. Max 3 files per request, 20 MB each.

Model, Settings, and Ollama

- `services/ollama_client.py` — `OllamaClient` wraps model listing, health checks, and generation calls.
- `services/model_preferences_store.py` and `services/postgres_store.py` persist per-use-case model routing.
- `services/config.py` — `Settings` and `load_settings` read environment values at startup.

Authentication, Users, and Permissions

- `services/auth_store.py` — file-store auth fallback.
- `services/postgres_store.py` — `PostgresAuthStore` for DB-backed auth.
- `api/main.py` — `require_current_user` and `require_admin` protect routes. `_resolve_registration_role` enforces bootstrap / admin / staff registration rules.
- `frontend/src/lib/auth.tsx` — auth state and session behavior.
- `frontend/src/lib/permissions.ts` — frontend admin / staff permission helpers.

Audit, Health, and Output Actions

- `services/audit_store.py` — file-store audit fallback.
- `services/postgres_store.py` — `PostgresAuditStore` for DB-backed audit events.
- `GET /api/health` and `GET /api/audit-events` in `api/main.py`.
- `frontend/src/components/OutputActions.tsx` — copy, text download, rich email output, and print / save-PDF behavior.

Dentrix Field Mapping

- `POST /api/dentrix/resolve-template-fields` in `api/main.py`.
- `services/dentrix_mapping.py` — `resolve_dentrix_template_fields` maps Dentrix-style claim data to template fields. Returns resolved fields, missing required fields, and missing optional fields.
- `docs/DENTRIX_FIELD_MAPPING_SPEC.json` — mapping specification.

Test Map

Test file	Coverage
<code>tests/test_api.py</code>	API behavior.
<code>tests/test_auth_store.py</code>	Auth store behavior.
<code>tests/test_autonomy_policy.py</code>	Grounding / autonomy policy behavior.
<code>tests/test_generation_security.py</code>	Generation security / path behavior.
<code>tests/test_verification.py</code>	Insurance verification normalization / grounding.
<code>tests/test_email_thread.py</code>	Email thread behavior.

tests/test_email_guardrails.py	Email guardrails behavior.
tests/test_document_pipeline.py	Upload / document pipeline behavior.
tests/test_template_runtime.py	Template runtime rendering.
tests/test_template_store.py	Template store behavior.
tests/test_dentrix_mapping.py	Dentrix mapping.
tests/test_ollama_client.py	Ollama client behavior.
tests/test_prompt_engine.py	Prompt rendering.
tests/test_field_dictionary_store.py	Field dictionary store.

12. Verification Checklist

Run Backend Tests

```
python3 -m unittest discover -s tests -p 'test_*.py' -v
```

Run Frontend Build

```
cd frontend npm run build
```

Check Docker Stack

```
docker compose ps
```

Check API

```
curl -sS http://localhost:8000/
```

Check Ollama

```
ollama ps ollama list
```

Manual Smoke Test

1. Open <http://localhost:3000>.
2. Bootstrap / log in as admin.
3. Confirm System Health loads.
4. Generate one denial letter.
5. Generate one insurance verification.
6. Generate one email exchange reply.
7. Edit a generated draft.
8. Save a personal template.
9. As admin, save / update a shared template.
10. Stop the app with `./stop.sh` and confirm models are offloaded with `ollama ps`.

13. Known Operational Constraints

- Ollama must be running on the host before generation works.
- The selected model must already be pulled locally.
- Vision-capable model behavior depends on the local model installed in Ollama.
- Insurance verification depends on local payer reference files. Unsupported payer data cannot be inferred reliably.
- Generated content must be reviewed by a human user before being sent, printed, or filed.
- The app is intended for local / private deployment, not public internet exposure without additional infrastructure hardening.

- The autonomy policy (grounding check) raises a 422 UNGROUNDED_FACTS error when the model output contains dates, IDs, procedure codes, or other verifiable values not present in the input. Provide missing facts via runtime fields or source documents.

14. Maintenance Notes

Adding a Payer

1. Add a text file under `data/payer_references/`.
2. Use lowercase filename formatting where spaces become underscores, for example `delta_dental.txt`.
3. Restart or refresh the app if the payer list does not immediately appear.

Adding or Changing a Prompt

1. Edit the relevant file under `prompts/`.
2. Confirm the prompt is registered in `services/prompt_registry.py`.
3. Run backend tests.
4. Run a manual generation test for the affected workflow.

Adding a Template Type

1. Admin creates the new type through the UI.
2. Add or update the shared canonical template for that type.
3. Confirm required variables are understandable to staff users.

Changing Model Routing

1. Pull the model locally with Ollama.
2. Confirm it appears in the Model Settings page.
3. Set the global or per-use-case preference.
4. Run a generation smoke test.

Managing the Field Dictionary

The field dictionary maps canonical runtime field keys (e.g., `patient_name`) to human-readable labels and alternate aliases. Use `PUT /api/field-dictionary/{field_key}` to add or update entries. Aliases are automatically applied during template rendering via `expand_runtime_field_aliases`.

15. Recent Changes

This section records significant changes to the codebase since the initial handoff draft.

Legacy File Cleanup (commit `c6ce15a`)

The following files were removed to clean up the repository:

- `app.py` — the original Streamlit prototype frontend (490 lines). The current frontend is the React + TypeScript application under `frontend/`.
- `scripts/migrate_file_store_to_postgres.py` — one-time migration script used to move data from the JSON file store to PostgreSQL. No longer needed.
- `.streamlit/config.toml` — Streamlit configuration, no longer relevant.

- HANDOFF .pdf, HANDOFF .md, README .md — superseded by this document.
- Multiple planning and discovery documents (PRD, TRD, RUNBOOK, implementation plans, agent notes, discovery questionnaire, development learnings, git push checklist) — consolidated into operational documentation.

Note for engineers: *If you need history from the original Streamlit app or migration script, use `git log` and `git show` to retrieve deleted content from earlier commits.*

End of Handoff Document